

Notas de Aula – PROLOG

Prof. Luiz Eduardo da Silva

PROLOG - Programming in Logic

É uma linguagem de programação centralizada em torno de um pequeno conjunto de mecanismos:

- Pesquisa por padrão (pattern-matching).
- Estrutura de dados baseada em árvore.
- Backtracking automático.

Este pequeno conjunto constitui uma estrutura poderosa e flexível de programação.

prolog é especialmente indicado para objetos - principalmente objetos estruturados - e as relações entre estes objetos. Muito usada como linguagem suporte em aplicações não numéricas e Inteligência Artificial (I.A.).

prolog foi uma idéia que nasceu na década de 70 para usar lógica com uma linguagem de programação. Os desenvolvedores desta idéia foram:

-**Robert Kowalski**: responsável pelo lado teórico

-**Maarten Van Emdem**: responsável pela demonstração experimental

-**Alain Comerauer**: responsável pela Implementação

Enquanto as linguagens convencionais usam um conceito de programação procedural (exemplo pascal), a linguagem prolog é uma linguagem declarativa.

Em que prolog pode ser usado

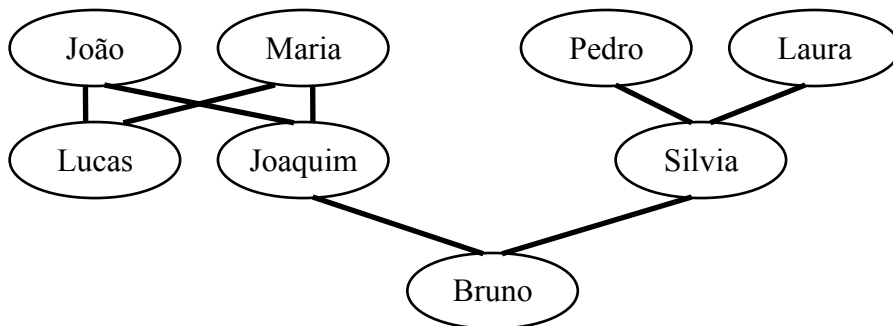
- Para controlar e monitorar processos industriais.
- Implementar Banco de Dados inteligentes.
- Traduzir linguagens para linguagens naturais (humana), ou de uma linguagem de programação para outra.
- Construir interfaces de linguagem natural.
- Construir Sistemas Especialistas.
- Construir pacotes para manipulação de símbolos que resolvem equações, diferenciais e integrais não definidas, demonstram teoremas.

Introdução à prolog

Esta seção apresenta as principais características de prolog através de um programa exemplo. Embora seja uma apresentação informal, aqui são introduzidos vários conceitos importantes.

Um programa exemplo: definindo relações familiares.

Prolog é uma linguagem para computação simbólica, não numérica. Ela é indicada para resolver problemas que envolvem objetos e relações entre estes objetos. A figura abaixo apresenta um exemplo de relações familiares:



O fato que João é pai de Joaquim pode ser escrito em prolog como:

pai(joao,joaquim) .

Aqui nos escolhemos **pai** como o nome da relação e **joao** e **joaquim** como os argumentos . Por razões que se tornarão mais claras mais tarde, nós escrevemos os nomes como joao e joaquim com a primeira letra minúscula e sem acento. A árvore genealógica acima pode ser definida pelo seguinte programa prolog:

```
pai(joao,joaquim). % joao e' pai de joaquim.
pai(joao,lucas) .
pai(pedro,silvia) .
pai(joaquim,bruno) .
mae(maria,joaquim) . % maria e' mae de joaquim.
mae(maria,lucas) .
mae(laura,silvia) .
mae(silvia,bruno) .
```

Este programa consiste de 6 (seis) cláusulas. Cada uma destas cláusulas declara um fato sobre as relações familiares: **pai** e **mãe**. Quando este programa for carregado num sistema Prolog, poderemos fazer algumas perguntas sobre estas relações familiares. Por exemplo: **pedro é pai de sylvia?** Esta questão pode ser transmitida para o sistema prolog digitando-se:

?- pai(pedro,silvia) .

Se prolog um fato correspondente no programa, responderá:

yes

Para a consulta:

```
?- pai(pedro,claudio) .
```

Prolog responderá:

```
no
```

Prolog não encontra nenhum fato no programa que diz que Pedro é pai de Cláudio, então responde não para esta questão.

Algumas questões mais interessantes podem ser perguntadas para o sistema Prolog. Por exemplo: quem é o pai de Sílvia?

```
?- pai(X,silvia) .
```

Prolog não responderá somente 'yes' ou 'no' desta vez. Prolog responderá qual é o valor de X que torna a questão acima verdadeira. Então a resposta será:

```
X = pedro
```

A questão 'quem é filho de João' pode ser perguntada à Prolog como:

```
?- pai(joao,X) .
```

Desta vez há mais de uma resposta para esta questão. Prolog apresenta a primeira solução:

```
X = joaquim;
```

Nós podemos agora pedir outras soluções. Se nós digitamos ; (ponto-e-vírgula), Prolog encontrará outra resposta:

```
X = lucas
```

Se nós requisitarmos outra solução, Prolog responderá com 'no' porque todas as respostas já foram encontradas.

Nosso programa pode responder a uma questão mais ampla. Quem é pai de quem? Outra formulação para esta questão é:

Encontre X e Y tal que X é pai de Y.

Isto é escrito em Prolog como:

```
?- pai(X,Y) .
```

Prolog agora encontrará todos os pares pai-filho. A solução será apresentada enquanto digitarmos ; (ponto-e-vírgula), e pára quando digitarmos . (ponto) ou nenhuma outra solução for encontrada. A resposta será:

```
X = joao  
Y = joaquim ;
```

```
X = joao  
Y = lucas ;
```

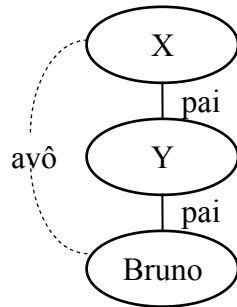
```
X = pedro
```

```
Y = silvia ;
```

```
X = joaquim;
```

```
Y = bruno .
```

Nosso programa Prolog pode responder a questões mais complicadas como: Quem é o avô de bruno? Como o nosso programa ainda não sabe sobre a relação familiar avô, esta consulta deve ser quebrada em dois passos, como ilustrado na figura abaixo:



1. Quem é o pai de bruno? Assuma que é algum Y.
2. Quem é o pai de Y? Assuma que é algum X.

Tal consulta pode ser feita ao sistema Prolog como a concatenação de duas questões simples:

```
?- pai(Y,bruno),pai(X,Y) .
```

A resposta será:

```
X = joao
```

```
Y = joaquim .
```

Nossa consulta pode ser lida como: Encontre X e Y que satisfaça aos dois requisitos: pai(Y,bruno) e pai(X,Y). Se mudarmos a ordem destes dois requisitos o significado lógico da consulta permanece o mesmo: pai(X,Y), pai(Y,bruno).

Exercícios:

1. Sejam pai e mãe as relações como definidas no programa exemplo anterior. Qual será a resposta de prolog para as seguintes questões:
 - pai(joao, X).
 - pai(X,joaquim).
 - pai(joao,X), pai(X,bruno).
2. Formule em prolog as seguintes consultas sobre as relações pai e mãe do programa anterior.
 - Quem é o pai de Bruno?
 - Maria tem filhos?
 - Quem é o avô de Bruno?

Estendendo o programa prolog para regras

Como uma extensão para o programa prolog anterior podemos escrever uma regra para as relações familiares: avô paterno, avô materno, avó paterna e avó materna. Em prolog estas relações podem ser definidas a partir das relações pai e mãe. Assim:

avo_paterno(X,Y) :- pai(X,Z) , pai(Z,Y) .

Esta regra pode ser lida como:

X é avo de Y se X é pai de Z e Z é pai de Y.

As regras especificam coisas que podem ser verdadeiras se uma condição é satisfeita. As regras tem duas partes:

A condição - lado direito da regra.

A conclusão - lado esquerdo.

Por exemplo :



As outras regras são:

avo_materno(X,Y) :- pai(X,Z) , mae(Z,Y) .

avoh_paterna(X,Y) :- mae(X,Z) , pai(Z,Y) .

avoh_materna(X,Y) :- mae(X,Z) , mae(Z,Y) .

Exercícios:

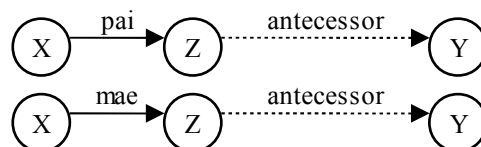
1. Traduza as seguintes afirmações em regras prolog:
 - Todos que tem filhos são felizes (defina uma relação de um argumento feliz).
 - X é irmão de Y se X e Y são filhos do mesmo pai e da mesma mãe.
2. Defina a relação neto a partir das relações pai e mãe.
3. Defina a relação anterior a partir das relações avo_materno, avo_paterno, avoh_paterna e avoh_materna.

Definição de regras recursivas

Vamos acrescentar a relação antecessor no programa de relações familiares. A definição toda pode expressa com algumas regras. A duas regras que definem o antecessor diretamente (pai e mãe) e as duas últimas que definem o antecessor de forma indireta (recursivamente). Assim:

```
antecessor(X,Y) :- pai(X,Y) .  
antecessor(X,Y) :- mae(X,Y) .  
antecessor(X,Y) :- pai(X,Z) , antecessor(Z,Y) .  
antecessor(X,Y) :- mae(X,Z) , antecessor(Z,Y) .
```

Definição recursiva da relação antecessor:



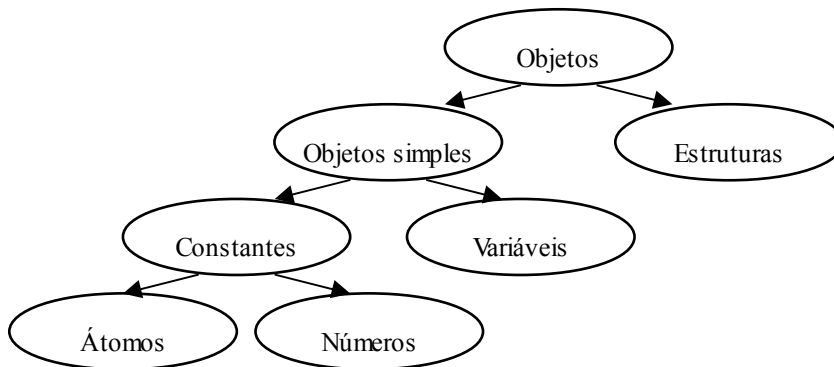
Exercício:

Defina regras para relação parente a partir da relação antecessor. Duas pessoas são parentes se tem um antecessor em comum.

Sintaxe e significado dos programas prolog

Apresentamos agora um pouco da sintaxe e semântica da linguagem prolog.

A figura abaixo mostra a classificação dos objetos prolog:



Objetos

Prolog pode reconhecer os tipos dos objetos sem que seja necessário uma declaração explícita como em outras linguagens, porque cada objeto tem uma forma diferente. Por exemplo, toda variável prolog começa com letra maiúscula.

Átomos e Números

Átomos podem ser escritos de três formas:

1. Strings de letras, números e underscore começando com letra minúscula. Ex: ana, nil, x25, x_25, etc.
2. Strings de caracteres especiais: Ex: <---->, ==>, etc.
3. Strings de caracteres entre aspas simples. Ex: 'Tom', 'América do Sul', etc.

Os números usados em prolog incluem os números inteiros e os números reais.

Variáveis

Variáveis são strings de letras, dígitos e underscore que começam com letra maiúscula ou underscore. Ex: X, Resultado, _x25, etc.

Quando uma variável aparece numa regra prolog somente uma vez, nós não precisamos inventar um nome para ela, basta usar a variável anônima representada por um símbolo underscore '_'. Exemplo:

```
tem_filho(X) :- pai(X,Y) .
```

Todo X tem filho se é pai de alguém. Esta regra pode ser escrita simplesmente como:

```
tem_filho(X) :- pai(X,_).
```

variável anônima

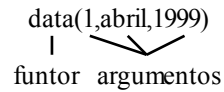
As variáveis anônimas podem aparecer em consultas. Por exemplo, suponha que queremos saber quais são os pais da família mas não estamos interessados nos nomes dos filhos. Podemos perguntar assim:

```
?- pai(X,_).
```

O escopo léxico de uma variável está na mesma regra, ou seja, toda variável prolog é local a regra em que ela aparece. Por exemplo, a variável X15 ocorre em duas regras, então temos duas variáveis diferentes. Mas cada ocorrência da variável X15 na mesma regra representa a mesma variável.

Estruturas

Objetos estruturados, objetos compostos ou simplesmente estruturas são objetos que contém vários componentes. Estes componentes podem ser estruturas. Por exemplo:



A data pode ser vista como uma estrutura com 3 componentes: dia, mês e ano. Para combinar os componentes num único objeto temos que escolher um **funtor**. Para este exemplo o funtor é data.

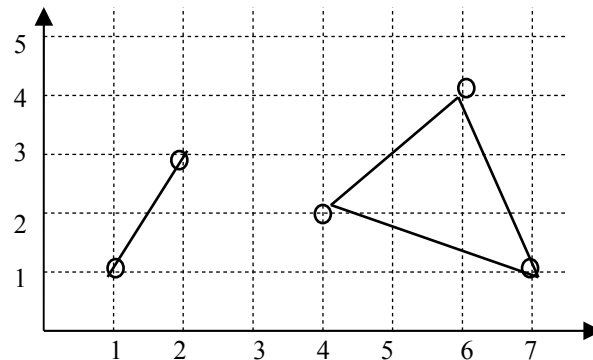
Todos os componentes neste exemplo são constantes (dois inteiros e um átomo). No entanto, podemos ter variáveis como componentes de uma estrutura, assim:

data(D,abril,1999).

Todo objeto em prolog é representado por uma árvore. Por exemplo, vamos definir estruturas prolog para definir alguns objetos geométricos simples. São eles:

- ponto no plano cartesiano - é definido por duas coordenadas.
- segmento - é definido por dois pontos.
- triângulo - é definido por três pontos.

Exemplo:



Em prolog:

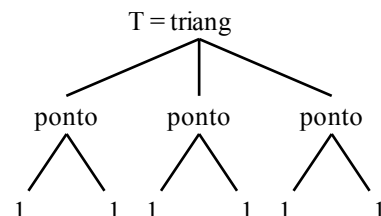
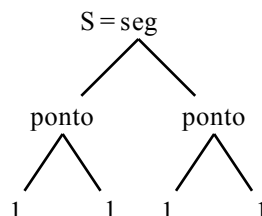
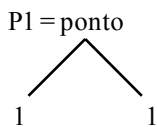
P1=ponto (1,1)

P2=ponto (2,3)

S=seg (P1,P2)=seg (ponto (1,1) ,ponto (2,3))

T=triang (ponto (4,2) ,ponto (6,4) ,ponto (7,1))

Estas estruturas representadas como árvores:



Listas

A lista é uma estrutura de dados simples largamente utilizada em programação não-numérica. Uma lista é uma seqüência de itens como: ana, pedro, paulo, joao. Esta lista pode ser representada em prolog como:

[ana, pedro, paulo, joao]

A lista vazia é representada por **[]**.

A lista não vazia contém duas partes:

- A cabeça da lista, o primeiro item da lista.
- A cauda da lista, o resto da lista sem o primeiro elemento.

Por exemplo, para a lista anterior:

ana = cabeça da lista.

[pedro, paulo, joao] = cauda da lista.

Alguns exemplos:

Lista	Cabeça	Cauda
[a,b,c]	a	[b,c]
[coisa]	coisa	[]
[]	indefinida	indefinida
[[1,3],[2,3,4],[]]	[1,3]	[[2,3,4],[]]

Variáveis são muitas vezes usadas para representar elementos desconhecidos e elementos genéricos de uma lista. Exemplo:

[A, B, pedro]

O primeiro e segundo elementos desta lista são genéricos ou desconhecidos e, por isso, estão representados pelas variáveis A e B. O terceiro elemento é conhecido e está representado pelo átomo pedro.

Um número indeterminado de elementos genéricos na cauda de uma lista pode ser representado por uma barra vertical (|) seguida de uma variável. Exemplo:

[rosa, cravo | X]

A barra vertical seguida da variável X indica que após o segundo elemento desta lista há uma quantidade ignorada de elementos desconhecidos ou genéricos.

A barra vertical pode ser traduzida por “resto da lista”. Isto faz com que [5,9,7|X] seja considerada como a lista cujo primeiro elemento é 5, o segundo elemento é 9, o terceiro elemento é 7 e o resto dos elementos é X.

Um padrão de lista muito importante é aquele cujo o primeiro elemento é uma variável e cujo resto da lista é também uma variável. Exemplos deste padrão são:

[X|Y]

[Primeiro|Resto]

[Topo|Cauda]

Tais listas representam qualquer lista com pelo menos um elemento. Considere para fixar

idéias, o caso de $[X|Y]$. Este padrão pode ser instanciado a lista $[ana]$ de um único elemento desde que X represente o valor 'ana' e Y represente a lista vazia. Num outro exemplo, se prolog tenta satisfazer a consulta:

```
placar([X|Y]).
```

e encontra o fato na sua base de conhecimento:

```
placar([0,1,2,3,4,5]).
```

X será instanciado ao valor 0 ($X = 0$) e Y será instanciado a lista $[1,2,3,4,5]$ ($Y = [1,2,3,4,5]$).

A próxima tabela mostra exemplos de casamentos de listas, onde as variáveis livres serão instanciadas aos valores da lista como no exemplo anterior.

Lista 1	Lista 2	Variáveis instanciadas
$[X, Y, Z]$	$[joao, maria, jose]$	$X=joao, Y=maria, Z=jose$
$[7]$	$[X Y]$	$X=7, Y=[]$
$[1, 2, 3, 4]$	$[X, Y Z]$	$X=1, Y=2, Z=[3, 4]$
$[1, 2]$	$[3, X]$	O casamento falha porque a cabeça das lista são diferentes.

Relação pertence

Vamos agora escrever um programa prolog para verificar se um elemento pertence a uma lista genérica qualquer. A primeira frase deste programa é:

```
pertence(X, [X|Y]).
```

Esta frase diz que X pertence a uma lista genérica se X é o primeiro elemento desta lista. Se X não é o primeiro elemento da lista, ele ainda assim pertence a lista se pertencer ao resto da lista. Esta segunda afirmação é escrita com a seguinte regra prolog:

```
pertence(X, [Primeiro|Resto]) :- pertence(X, Resto).
```

Testemos o programa com algumas consultas:

```
?- pertence(a, [b,c,d,a,f,g]).
```

```
yes
```

```
?- pertence(ana, [rosa, cravo, violeta]).
```

```
no
```

```
?- pertence(E, [1,2,3,4]).
```

```
E = 1;
```

```
E = 2;
```

```
E = 3.
```

```
yes
```

Observe que a última consulta tem várias respostas. Teclando ponto-e-vírgula no final de cada uma, fizemos o computador encontrar as três primeiras.

Relação concatenação

Para concatenar listas nos definiremos a relação:

ap(L1, L2, L3)

Onde L1 e L2 são duas listas, e L3 é o resultado da concatenação de L1 e L2. Por exemplo:

ap([a,b], [c,d], [a,b,c,d]).

é verdade, mas

ap([a,b], [c,d], [a,b,a,c,d]).

é falsa.

Na definição da relação para concatenação de lista ap, nós precisamos tratar dois casos, dependendo do primeiro argumento L1:

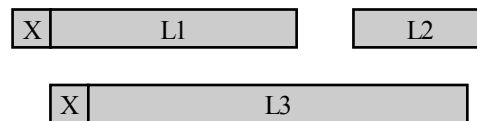
1. Se o primeiro argumento é uma lista vazia então o segundo e o terceiro argumento precisam ser a mesma lista. Isto é definido pelo seguinte fato prolog:

ap([], L, L).

2. Se o primeiro elemento não é uma lista vazia, então ele tem cabeça e cauda. Seja o primeiro argumento a seguinte lista genérica [X|L1]. O resultado da concatenação desta lista com a lista L2 é uma lista [X|L3], onde L3 é a concatenação de L1 (cauda da primeira lista), com L2 (segunda lista). Em prolog, esta regra é escrita como:

ap([X|L1], L2, [X|L3]) :- ap(L1, L2, L3).

A figura seguinte ilustra a concatenação da lista [X|L1] com alguma lista L2:



Este programa pode ser usado para concatenar as listas dadas, por exemplo:

?- ap([a,b,c], [d,e,f], L).

L = [a,b,c,d,e,f]

Entretanto podemos usar esta relação ap na direção inversa para decompor uma dada lista em duas listas, assim:

?- ap(L1, L2, [1,2,3]).

L1 = []

L2 = [1,2,3];

L1 = [1]

L2 = [2,3];

L1 = [1,2]

L2 = [3];

L1 = [1,2,3]

L2 = [];

no

É possível decompor a lista [1,2,3] de quatro formas distintas, todas as formas foram encontradas por nosso programa prolog graças ao mecanismo automático de retrocesso para encontrar outras respostas possíveis para uma consulta, denominado *backtracking*.

Nos podemos usar nosso programa ainda para verificar certos padrões numa lista. Por exemplo, nos podemos encontrar os meses que precedem e sucedem um mês dado através da seguinte consulta:

```
?- ap(Antes,[mai|Depois],
      [jan,fev,mar,abr,mai,jun,jul,ago,set,out,nov,dez]).
Antes = [jan,fev,mar,abr]
Depois = [jun,jul,ago,set,out,nov,dez].
```

A relação pertence anterior pode ser elegantemente programada usando o predicado `ap` com a seguinte regra:

```
pertence1(X,L) :- ap(L1,[X|L2],L).
```

Esta regra diz que `X` pertence a uma lista `L`, se `L` pode ser decomposta em duas listas tal que a segunda lista tenha o elemento `X` como cabeça. Esta regra fica melhor escrita com o uso de variável anônima, assim:

```
pertence1(X,L) :- ap(_, [X|_], L).
```

Relação apaga

Apagar um item `X` de uma lista `L` pode ser programado com a seguinte relação:

```
apaga(X, L, L1).
```

onde a lista `L1` é a lista `L` com o elemento `X` removido.

A relação `apaga` é similar a relação `membro da lista`. De novo temos que pensar recursivamente em dois casos:

1. Se `X` é a cabeça da lista então o resultado é a cauda da lista.
2. Se `X` está na cauda da lista então ela será retirada de lá.

Fica assim então:

```
apaga(X, [X|CAUDA], CAUDA).
apaga(X, [Y|Z], [Y|Z1]) :- apaga(X, Z, Z1).
```

OBS: Se houver várias ocorrências de `X` na lista `L` o predicado `apaga`, apagará uma ocorrência por *backtracking*. Por exemplo:

```
?- apaga(a,[a,b,a,a], L).
L = [b,a,a];
L = [a,b,a];
L = [a,b,a];
no
```

O predicado `apaga` também pode ser usado na direção inversa, para inserir um item em algum lugar de uma lista. Assim:

```
?- apaga(4, L, [1,2,3]).
L = [4,1,2,3];
L = [1,4,2,3];
L = [1,2,4,3];
```

L = [1,2,3,4] ;

no

Exercícios:

1. Defina os dois predicados prolog. listapar (Lista) - que é verdadeiro quando o número de elementos da lista for par. E listaimpar (Lista) - que é verdadeiro quando o número de elementos da lista for impar.
2. Defina a relação reversa (Lista, ListaReversa). Por exemplo: reversa ([a,b,c,d], X) X=[d,c,b,a].
3. Defina a relação shift (Lista1, Lista2) tal que L2 seja a lista uma rotação para a esquerda. Assim: shift ([1,2,3], L2) então L2 = [2,3,1]
4. Defina a relação traducao (Lista1, Lista2) para traduzir uma lista de números de 0 a 9 para a lista dos números por extenso. Exemplo: traducao ([1,3,9], L2), L2 = [um, tres, nove].

Usar o seguinte predicado auxiliar:

significa (0, zero).

significa (1, um).

(...)

significa (9, nove).

5. Escreva um predicado em prolog que apaga de uma lista de inteiros o elemento de uma posição relativa na lista. Exemplo: apaga (3, [6,7,8,9,1,2,3], L), L = [6,7,9,1,2,3]. Observe que foi apagado o terceiro elemento da lista.
6. Desenvolva um predicado em prolog que troca a primeira ocorrência de um certo inteiro numa lista de inteiros dando como resultado uma lista com um elemento alterado. Exemplo: altera (3, 4, [1,2,3,4], L), L = [1,2,4,4]
7. Implemente um predicado de nome **menor** que encontre o menor elemento numa lista de inteiros. Exemplo: menor ([1,5,4,3,0,9,8,7], X), X = 0
8. Escreva um predicado em prolog que encontra o maior valor numa lista de inteiros. Exemplo: maior ([1,2,3,9,5,4,6], Y), Y = 9.
9. Defina um predicado denominado **gera2x** que gera uma lista de dobro do numero que é passado como objeto até zero (0). Exemplo: gera2x (5, L), L = [10, 8, 6, 4, 2, 0].
10. Desenvolva um predicado prolog que a partir de uma posição relativa dentro de uma lista de inteiros, dê como resultado a lista com os elementos até a posição menos 1. Exemplo: trunca (3, [1,4,5,6,7], L), L = [1,4].
11. Escreva um predicado que a partir de um valor inteiro gere uma lista de cubos dos valores. Assim: cubo (3, C), C = [27, 8, 1, 0].

O corte (!)

Em algumas situações é preciso impedir que o programa prolog volte atrás em uma decisão

tomada (*backtracking*). Isto pode ser feito usando um predicado que se chama corte e é representado por um ponto de exclamação.

Representação:

`<a> :- <b>, ..., <c>, !, <d>, <f>, ...`

`<a> :- <g>, <h>, <i>, ...`

Este corte é usado para interromper backtracking em dois casos:

- Nas condições anteriores a ele na mesma regra.
- Nas regras abaixo daquela em que ele aparece.

Seja a regra que não contenha corte com o seguinte formato:

`<a> :- <b>, <c>, <d>.`

Tal regra diz, na interpretação declarativa, que: `<a>` é verdadeiro se `<b>`, `<c>` e `<d>` forem verdadeiros. Isto equivale a dizer que se `<b>`, `<c>` e `<d>` forem verdadeiros então `<a>` é verdadeiro.

O corte pode ser traduzido por “quando e só quando”. Assim sendo, uma frase com o formato:

`<a> :- <b>, <c>, !, <g>, <h>.`

deve receber a interpretação do tipo, se `<b>` e `<c>` são verdadeiros então `<a>` é verdadeiro quando e só quando `<g>` e `<h>` são verdadeiros.

Programa Exemplo:

```
add(X,L,L) :- in(X, L), !.  
add(X,L,[X|L]).  
  
in(X,[X|_]) :- !.  
in(X,[_|L]) :- in(X,L).
```

Exercício:

1. Reescreva o programa para apagar um elemento de uma lista usando o corte.

Predicado Findall

Findall é um predicado pré-definido de prolog gera uma lista com todas as alternativas encontradas que satisfaça a uma consulta.

Programa exemplo:

```
aluno(pedro,9).  
aluno(joao,10).  
aluno(joaquim,9).  
  
teste_findall :-  
    findall(X,aluno(X,_),L),  
    write(L),  
    nl.
```

Com a consulta:

?- teste_findall.

prolog responderá:

```
[pedro, joao, joaquim]
```

```
yes
```

Exercícios:

1. Modifique o programa anterior para que seja montada uma lista somente com os nomes dos alunos com 9 anos de idade.
2. Escreva um predicado para encontrar a média de idade dos alunos usando os predicados findall, total_lista e nro_alunos.

Operadores

Prolog admite que qualquer funtor de dois argumentos seja infixo. Isto significa que o funtor pode ser posicionado entre os dois argumentos. Por exemplo, nós podemos definir o átomo tem como um operador infixo e então escrever os seguintes fatos:

```
pedro tem casa.
```

```
pedro tem carro.
```

Estes fatos são equivalentes a:

```
tem(pedro, casa).
```

```
tem(pedro, carro).
```

O programador pode definir este novo operador da seguinte forma:

```
:- op(600,xfx,tem).
```

Isto diz a prolog que queremos usar o átomo tem como operador, cuja precedência é 600 e ele é do tipo xfx, que é um tipo de operador infixo. Esta forma sugere que o operador, denotado por 'f', está entre os dois argumentos denotados por 'x'.

Programa exemplo:

```
:-op(600,xfx,tem).  
pedro tem casa.  
pedro tem carro.
```

Aritmética

Prolog é uma linguagem para computação simbólica, sendo assim a necessidade para cálculos numéricos é comparativamente modesta. Alguns dos operadores básicos para operações aritméticas em prolog são:

+ adição

- subtração

* multiplicação

/ divisão

mod módulo, resto da divisão inteira.

Para fazer cálculos em prolog, lançamos mão de predicados com a declaração 'is'. Esta declaração é infixa e, portanto, deve ser colocada entre dois objetos. O primeiro objeto é uma variável e o segundo é uma expressão aritmética. O efeito do operador 'is' é realizar o cálculo indicado pela expressão e colocar o valor encontrado na variável. Para compreender melhor o que

foi dito, examinemos a pergunta abaixo e a resposta que ela recebeu.

?- X is 2+5*9.0/4.

X = 13.25.

yes

A expressão '2+5*9.0/4' foi calculada e o valor obtido ficou associado à variável X.

Programa exemplo:

```
nro_elementos([],0).  
nro_elementos([_|Y], N) :-  
    nro_elementos(Y,NY),  
    N is NY + 1.
```

Os operadores relacionais de prolog são:

X > Y X é maior do que Y

X < Y X é menor do que Y

X >= Y X é maior ou igual a Y

X <= Y X é menor ou igual a Y

X \= Y X é diferente de Y

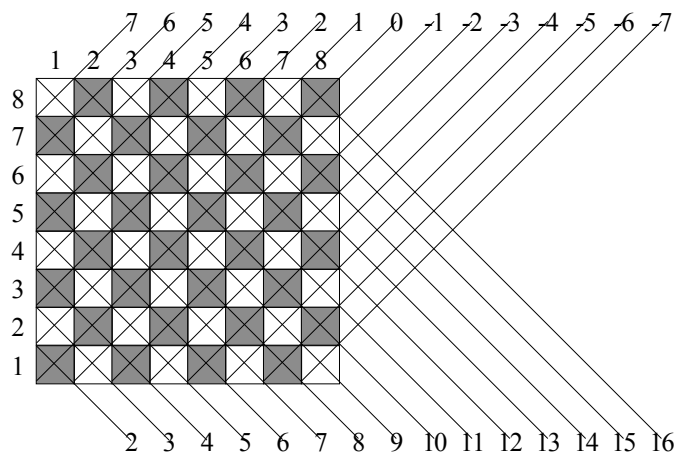
Programa exemplo:

```
mdc(X,X,X).  
mdc(X,Y,D) :-  
    X < Y,  
    Y1 is Y - X,  
    mdc(X,Y1,D).  
mdc(X,Y,D) :-  
    Y < X,  
    mdc(Y,X,D).
```

Problema das 8-rainhas

O problema clássico em inteligência artificial, chamado de problema das 8-rainhas, consiste em posicionar 8 rainhas de xadrez num tabuleiro 8x8 de modo que uma rainha não possa capturar outra. Lembrando que uma rainha captura outra se ambas estiverem numa mesma diagonal, linha ou coluna do tabuleiro.

Representação:



Para resolver este problema, precisamos numerar todas as linhas, colunas, diagonais superior e inferior. Escolhida a linha e a coluna para posicionar uma rainha, temos que verificar se as diagonais correspondentes não estão ocupadas por outra rainha.

Programa:

```
/*+-----+
| SOLUCAO DO PROBLEMA DAS 8 RAINHAS          |
| Por Luiz Eduardo da Silva                  |
+-----+*/
solucao( S) :-
    resolve( S, [1,2,3,4,5,6,7,8],
             [1,2,3,4,5,6,7,8],
             [7,6,5,4,3,2,1,0,-1,-2,-3,-4,-5,-6,-7],
             [2,3,4,5,6,7,8,9,10,11,12,13,14,15,16]).

resolve( [], [], _, _, _).
resolve( [C|LC], [L|LL], CO, DS, DI) :-
    apaga( C, CO, CO1),
    NS is L - C,
    apaga( NS, DS, DS1),
    NI is L + C,
    apaga( NI, DI, DI1),
    resolve( LC, LL, CO1, DS1, DI1).

apaga( A, [A|B], B).
apaga( X, [Y|Z], [Y|Z1]) :-
    apaga( X, Z, Z1).
```

São 92 configurações do tabuleiro de xadrez que resolvem este problema. O programa anterior quando consultado sobre as soluções do problema `solucao(S)`, responde da seguinte forma:

(...)

S=[7,3,8,2,5,1,6,4];

S=[7,4,2,5,8,1,3,6];

S=[7,4,2,8,6,1,3,5];

S=[7,5,3,1,6,8,2,4];

S=[8,2,4,1,7,5,3,6];

S=[8,2,5,3,1,7,4,6];

S=[8,3,1,6,2,5,7,4];

S=[8,4,1,3,6,2,7,5];

no

?-

	1	2	3	4	5	6	7	8
8					R			
7							R	
6		R						
5						R		
4			R					
3	R							
2				R				
1								R

A lista `S=[8,4,1,3,6,2,7,5]`, por exemplo, tem o seguinte significado:

A primeira rainha deve estar na linha 1 e coluna 8. A segunda rainha na linha 2, coluna 4. A terceira rainha na linha 3, coluna 1. A quarta rainha na linha 4, coluna 3. A quinta rainha na linha 5, coluna 6. A sexta rainha na linha 6, coluna 2. A sétima rainha na linha 7 e coluna 7. E a oitava e última rainha deve estar na linha 8 e coluna 5. Assim: